

NDU Project — Daily Development Report

To	Freebuff
From	Chungu Chama — NDU Project
Date	4 September 2026
Subject	Development progress report — work completed on 3 September 2026
Project	NDU Project Management Platform
Repository	https://github.com/CHAMA18/Ndu_Project
Environment	Staging (staging.admin.nduproject.com) — branch staging

1. Purpose

This report records the development work carried out on the NDU Project on **Wednesday, 3 September 2026**. It sets out the work programme agreed following the product-owner review of the staging environment, the items delivered against that programme, the verification performed, and the matters remaining open.

2. Background and work programme

The product owner conducted a review of the staging environment on 3 September 2026 (voice note, “New Recording 6”, transcribed and recorded in docs/voice_notes/2026-09-03-recording-6-transcript.md). The review distilled eight requirements for the platform. The governing principles were:

1. **The “skeleton” must work as core functionality, without AI.** The core flow — WBS breakdown (→) Schedule (→) Cost; Cost + Schedule (→) Controls; and management based on the Controls — is the foundation of the application. All other modules (people, quality, orders, procurement) feed that skeleton. If the skeleton does not work, the application serves no purpose.
2. **Manual cost estimation is core.** Every work package appearing in the Schedule must be priceable at the smallest (leaf) WBS level. Entering a value against a work package is core functionality and must not require AI. AI is acceptable only for *suggesting* prices and for *checking the site for missing items* (a minimal AI cost).
3. **Data movement between sections is core.** Copying or carrying a topic from one phase to another (e.g. initiation (→) planning) is core functionality and must not depend on AI “grabbing things”.
4. **AI must read the latest version of a topic.** Where a topic exists in several phases, AI should use the most recent phase’s version, falling back to an earlier phase only when no later version exists.
5. **Schedule costs must flow into the Cost Estimate** rather than being re-entered, and costs not represented in the Schedule (project manager, structural engineer, permits, government fees, architect, etc.) are added as additional estimate-level costs on top of the work-package costs.

The day’s implementation session executed these requirements across four workstreams (WS1–WS3 and a fourth corrective round), as detailed below.

3. Work completed

3.1 WS1 — Manual cost estimation at leaf-WBS level (core, no AI)

- Added pure coverage helpers (lib/wbs/utils/wbs_cost_coverage.dart) that collect leaf work packages, detect which leaves are linked to a cost line or priced (by costLineIds or by WBS reference), and report the leaves that remain unpriced — i.e. the smallest level at which cost is estimated.
- The **Cost by WBS** tab (used in both the WBS module and the Cost Estimate module) now includes a “**Work Packages Without Cost (N of M)**” section listing every unpriced leaf work package. Each row carries an **Add cost** action that opens the manual cost-line dialog pre-linked to that WBS node; the user types a quantity × rate or a lump total and the line is stored with a bidirectional link to the work package. No AI is involved.
- The tab now observes both the cost and WBS providers, so coverage updates immediately after any manual entry.

3.2 WS2 — AI and downstream flows read the latest version of a topic

- Added a reusable resolver (`lib/utils/latest_phase_content.dart`) that implements the product rule: a later phase wins; an earlier phase is authoritative only when nothing later exists; placeholders count as empty. It exposes pure helpers (`resolveLatestNonEmpty`, `resolveLatestPerArea`, `buildLatestContextSummary`) ready for any AI context builder that scans multiple phases.
- Audited the cross-phase write paths (e.g. WBS seeding from initiation, front-end-planning procurement/contract seeding). These already honour latest-wins behaviour — idempotent and seed-only-if-empty — and never overwrite a later phase with earlier data.

3.3 WS3 — Skeleton hardening (WBS (→) Schedule (→) Cost (→) Controls)

- Fixed a cold-start data-loss bug in which saved cost estimates could not be reloaded: `EstimateClass.name` is overridden to the display label (e.g. “Budget Authorization”), while reload looked the class up by identifier only, causing the whole estimate to fail to deserialize silently. Reload now accepts the identifier or the display label and defaults safely.
- Cost Estimate persistence now round-trips the **Basis of Estimate (BOE)** — scope basis, assumptions/constraints/exclusions, methodology, data sources and accuracy range — and **stakeholders**, which were previously dropped on save. Parse failures degrade to safe defaults instead of aborting the load.

3.4 Round 2 — Schedule-to-cost flows and pricing coverage KPI

- **One-click pull of scheduled purchases into the Cost Estimate.** The Schedule module now shows a status strip with a “**Pull N into Cost Estimate**” action (`CostEstimateProvider.pullScheduledPurchases`) that creates procurement cost lines from the purchases already visible in the Schedule — carrying the WBS reference, starting at \$0 until priced, and idempotent (no double counting). This is core data movement, not AI.
- **Costs entered directly on schedule work packages.** Each activity row in the Schedule Builder carries a cost action: *Add* on leaf work packages and *Edit* where a linked cost line already exists, with the activity stamped with the resulting cost-line id.
- **Pricing coverage KPI.** The Cost Dashboard now shows a **Work-Package Pricing Coverage** card — the percentage of leaf WBS work packages that are priced, with a progress bar, priced/total counts, and a hint when the WBS is not yet set up.

3.5 Round 3 — Latest-version AI context and schedule/cost finishing touches

- **Latest-phase resolver wired into AI context builders.** A new `ProjectDataHelper.buildLatestTopicOverlay` renders a “Latest topic versions” overlay (e.g. Goals in initiation vs planning; Technology and Infrastructure in initiation vs planning), and both `ExecutionPhaseAiSeed.buildContext` and `LaunchPhaseAiSeed.buildFullPhaseDependencyContext` append it — so AI prompts always read the most recent phase of a recurring topic and never a stale earlier copy.
- **Leaf-coverage indicator** added to the WBS module’s Cost-by-WBS header (cost lines · total · linked % · leaf work packages priced %).
- **“Price them now” walkthrough.** After pulling purchases, the confirmation offers a walkthrough that opens the manual cost dialog for each newly pulled line in order (cancelling or skipping advances to the next).
- **Priced/unpriced confirmation on Schedule rows.** Activity rows now carry a live badge — green “Cost: \$X” when the linked cost line is priced, amber “Cost: not priced yet” when linked but still unpriced — updating immediately after a pull or a manual entry.

3.6 Round 4 — Fix for blank Schedule Builder content

- **Reported symptom:** the Schedule Builder tab rendered without its content (“missing schedule items”) once synced planning content populated the tree.
- **Root cause:** the Project Timeline card’s activity rows used a stretch cross-axis alignment with only a minimum height inside the Builder’s vertically unbounded scroll view; with no bounded extent the stretch row threw a `RenderFlex` layout exception that blanked the whole tab.
- **Fix:** each timeline activity row now has a bounded height, resolving the stretch row; long activity names ellipsize on a single line, consistent with the Activity Tree rows. An `IntrinsicHeight` approach was trialled first but is incompatible with the bar area’s `LayoutBuilder`, so the bounded-height fix was adopted.

- Verified with widget reproductions covering both a populated synced tree and a root-only tree; both now render without exceptions.

4. Key deliverables

New modules:

- lib/wbs/utils/wbs_cost_coverage.dart — leaf work-package coverage helpers
- lib/utils/latest_phase_content.dart — latest-phase content resolver
- lib/schedule/utils/schedule_purchase_cost.dart — pullable-purchase detector

Modified modules:

- Cost Estimate: models/cost_estimate_models.dart, providers/cost_estimate_provider.dart, screens/cost_estimate_module_screen.dart, widgets/add_line_dialog.dart
- Schedule: screens/builder_screen.dart, screens/schedule_module_screen.dart
- Shared: widgets/cost_by_wbs_tab.dart, screens/project_dashboard_screen.dart
- AI context: utils/project_data_helper.dart, utils/execution_phase_ai_seed.dart, utils/launch_phase_ai_seed.dart

New test suites:

- test/wbs_cost_coverage_test.dart
- test/latest_phase_content_test.dart
- test/cost_estimate_boe_persistence_test.dart
- test/cost_estimate_pull_purchases_test.dart
- test/schedule_purchase_cost_test.dart
- test/latest_topic_overlay_test.dart
- test/schedule_row_cost_badge_test.dart

5. Verification and quality

- flutter analyze is clean on all touched files.
- The related unit and widget suites — cost, WBS, schedule, lifecycle and routing — pass: 47 tests reported at the close of the first three workstreams, with a further 11+ assertions and additional widget suites added during the later rounds of the session.
- The Schedule Builder regression was verified with widget reproductions before and after the fix.

6. Outstanding items and next steps

1. **Commit and deploy to staging.** The 3 September changes are implemented in the local staging workspace and are to be committed and pushed to staging.
2. **Notification protocol.** Per the process note in the review, updates pushed to staging are to be announced to the owner at the time of the push; this was missed in the previous round and is now an agreed standing practice.
3. **Owner validation.** An end-to-end walkthrough on staging to confirm the skeleton — WBS breakdown (→) Schedule (→) Cost; Cost + Schedule (→) Controls; management based on Controls — works as core functionality, ahead of the next review session.
4. **Continued build-out.** People, quality, orders and procurement modules continue to feed the skeleton, and the “minimal AI” checks for missing items can now be built on the coverage helpers delivered in this session.

Report compiled by:

Chungu Chama NDU Project

Reference: docs/voice_notes/2026-09-03-recording-6-transcript.md — product-owner review and implementation log for 3 September 2026.